

Game Project 2 Documentation

Introduction:

This game is designed as a mobile game application. It takes inspiration from the popular mobile game called *Doodle Jump*. My game *Robin's Ascent* is similar but themed after the Robin Hood storyline. The player's character is Robin Hood who shoots arrows at Nottingham's castle guards. As the guards are killed they drop gold coins, the player's objective is to collect these coins before reaching a goal height. If the player reaches this height with the required gold, they reach the next level where they have to collect more gold coins and reach a higher height. This introduces player progression where there is a clear goal to help with player engagement. The controls are the left/right arrow keys or A/D keys and the space bar to shoot the arrows (which shoot at a 45 degree upward angle depending on which way the player is facing).

Summary of Scripts:

- ArrowScript:
 - This script is used to generate an arrow sprite object when the player presses the space bar and shoots the arrow at a 45 degree angle in an upward direction to the left/right depending on which way the player is facing
- DestroyOffScreen:
 - This is important because it is attached to the arrow, enemy, and platform objects and destroys them when they are off screen. This ensures the objects don't stay alive when they're no longer needed and frees up resources (otherwise the game would slow and crash eventually)
- GameManager:
 - This is where all the magic happens. This script keeps track of the player's progression, advances the player to the next level when appropriate, handles game over states, updates the UI, and more.
- Platform Spawner:
 - This script spawns new platforms left/right randomly as the player advances upward along with the enemies that occasionally spawn on the platforms
- PlayerController:
 - This script handles player movement via inputs, also handles platform collisions which results in jumping upward. It also handles what happens

when the player moves off screen to the left/right - screen wrapping. This script also handles shooting arrows

- **CoinScript:**
 - Handles what happens when the player collides with a coin and collects it
- **EnemyScript:**
 - Handles what happens when the player's arrow collides with the enemy object thus spawning a gold coin in its place
- **MainMenu:**
 - Defines methods that are attached to the play and quit buttons on the menu, thus allowing the user to play or quit the game
- **PauseMenu:**
 - Defines what happens when the player pauses the game and ensures the game is stopped allowing the game to resume when the player presses resume or hits ESC again
- **SettingsMenu:**
 - Defines control and manages all settings changes between the UI and the associated attributes

Description of Scenes:

- **Main Menu Scene:**
 - This scene is used to display and layout the main menu where the game begins and the player can quit the game from. Settings can also be accessed from here.
 - Key Game Objects:
 - Canvas: this defines the interactive buttons where the player can play/quit/access settings
 - MainMenu: this is where the MainMenu script is attached allowing for the management of the game
- **Game Scene:**
 - This scene is where the gameplay happens. This scene is designed as such where the player (Robin Hood) jumps upward off tree branches running into the enemy guards until he reaches the height goals with the collected coins he obtains from these guards.
 - Key Game Objects:
 - Main Camera: this ensures the game view follows the player as he ascends the levels

- Robin: this is the playable character that defines how the player looks via the sprite and has the PlayerController script attached handling all the player interactions with the game world
- GameManager: this is the object that has the associated script to manage the game state and progression
- PlatformSpawner: this object manages the platforms and enemies to be spawned as the player progresses
- PauseMenuManager: this object allows the player to pause the game and UI overlay

Identify Satisfied Project Requirements:

- Player Input:
 - Input via left/right arrow keys or A/D keys for left/right movement and space bar for shooting arrows
- Saving/Loading:
 - TBI (To Be Implemented)
 - Altho, audio and fullscreen settings have been saved to be persistent throughout the game
- Sound:
 - Basic audio is generated automatically and level can be controlled via settings screen
- Settings:
 - Settings screen has been added in the main menu to control audio
- Animation:
 - As the player moves left/right the Robin Hood sprite flips direction to illustrate the direction the player is moving left/right
- Main Menu, Gameplay Scene, Pausing
 - The game starts on the main menu where they can play/quit/change settings
 - The gameplay runs in its own game scene as described above
 - Pausing can be done at any time with the ESC key -> Main Menu -> Quit
- Gameplay Progression:
 - At the top of the screen there are values that indicate the number of gold coins / gold coins the player has collected and the player's current height / goal height (this is the height the player has to reach with the required gold coins in order to progress to the next level where these goals get more difficult). This progression continues until the player falls and resets the game
- Win/Lose Conditions:

- If the player falls below the bottom of the screen he loses the game and starts over. If the player reaches the height goal with the goal coins he wins the current level and progresses to the next level; however, there is game ending win condition currently

Known Bugs or Incomplete Features:

- (Bug) Platform Collision:
 - Rarely the player can get stuck in a platform branch (which can easily be unstuck)
- (Feature) Enemy doesn't shoot back or hit player when colliding (this would be a cool feature that can add another level of difficulty)
- (Feature) More interesting level design where Robin is not just bouncing off trees but other environment elements (i.e. houses, castle structure, etc...)

Reference Material:

- Chat-GPT: Assisted moderately in writing scripts and generating sprites
 - All scripts and sprites were reviewed and modified (from slightly to extensively) in my project